

7. Revisitando o problema de aprendizagem

Em aprendizado supervisionado, o objetivo principal é criar uma aproximação $h : \mathcal{X} \rightarrow \mathcal{Y}$ para uma função $f : \mathcal{X} \rightarrow \mathcal{Y}$ a partir de amostras $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, onde cada exemplo de treinamento é uma amostra independente proveniente de $\mathbb{P}_{\mathbf{x}, y}$ e y_n é uma observação (possivelmente ruidosa) de $f(\mathbf{x}_n)$. No caso de regressão, poderíamos usar vários métodos distintos para construir h . Alguns exemplos que já vimos são k -NN, regressão linear e redes neurais RBF. Além disso, podemos alterar drasticamente o comportamento desses modelos alterando hiper-parâmetros. Isso nos leva às duas perguntas estruturantes desse capítulo:

1. Qual é a melhor classe de modelos (e.g., regressão linear, rede RBF) para cada problema?
2. Como escolher os melhores hiper-parâmetros para cada classe de modelos?

Por simplicidade, vamos considerar o caso em y_n 's não são ruidosos para maior parte da nossa discussão, isso é, $y_n = f(\mathbf{x}_n)$. No entanto, todas as conclusões desse capítulo se aplicam ao caso geral e é fácil adaptar as derivações para considerar o ruído.

7.1 Dilema viés-variância

De maneira geral, aprendemos h usando o conjunto de treino $\mathcal{D}$ com N amostras (i.e., $|\mathcal{D}| = N$), mas queremos que h apresente boa performance em novas observações de $\mathbf{x} \sim \mathbb{P}_{\mathbf{x}}$. Em outras palavras, gostaríamos de um modelo que minimiza uma função de perda ℓ média $\mathbb{E}_{\mathbf{x}}[\ell(h(\mathbf{x}), f(\mathbf{x}))]$. Esse valor é comumente chamado de erro de generalização.

Para a seguinte discussão, assuma que ℓ é o erro quadrático — i.e., $\ell(y, \hat{y}) = (y - \hat{y})^2$. Portanto o erro de generalização é dado por $\mathbb{E}_{\mathbf{x}}[(h(\mathbf{x}) - f(\mathbf{x}))^2]$. Como h é o resultado de um processo de aprendizado, ele depende diretamente dos exemplos em $\mathcal{D}$. Por exemplo, se estamos fazendo regressão linear, h pode ser obtida aplicando via máxima verossimilhança. No entanto, podemos analisar h de maneira mais geral, sem se prender aos valores específicos em $\mathcal{D}$. Para tal, calculamos o valor esperado do erro tratando $\mathcal{D}$ como uma variável aleatória. Isto é, calculamos uma média ponderada sobre todos os possíveis bancos de dado de tamanho N — com pesos dados por $\mathbb{P}_{\mathbf{x}, y}$. Usando a linearidade do operador de esperança e a independência de $\mathcal{D}$ e $\mathbf{x}$, segue que:

$$\begin{aligned}\mathbb{E}_{\mathbf{x}, \mathcal{D}} [(h_{\mathcal{D}}(\mathbf{x}) - f(\mathbf{x}))^2] &= \mathbb{E}_{\mathbf{x}, \mathcal{D}} [h_{\mathcal{D}}(\mathbf{x})^2 + f(\mathbf{x})^2 - 2h_{\mathcal{D}}(\mathbf{x})f(\mathbf{x})] \\ &= \mathbb{E}_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})^2]] + \mathbb{E}_{\mathbf{x}} [f(\mathbf{x})^2 - 2\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})]f(\mathbf{x})]\end{aligned}$$

Naturalmente, podemos somar $\mathbb{E}_{\mathbf{x}, \mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})]^2 - \mathbb{E}_{\mathbf{x}, \mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})]^2$ à última linha para obter:

$$\mathbb{E}_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})^2] - \mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})]^2] + \mathbb{E}_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})]^2 + f(\mathbf{x})^2 - 2\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})]f(\mathbf{x})],$$

Finalmente, aplicando a definição de variância — $\text{Var}_z[\psi(z)] = \mathbb{E}_z[\psi(z)^2] - \mathbb{E}_z[\psi(z)]^2$ — ao primeiro termo da equação acima e reorganizando o segundo termo, concluímos:

$$\mathbb{E}_{\mathbf{x}, \mathcal{D}} \left[(h_{\mathcal{D}}(\mathbf{x}) - f(\mathbf{x}))^2 \right] = \underbrace{\mathbb{E}_{\mathbf{x}}[\text{Var}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})]]}_{\text{Variância}} + \underbrace{\mathbb{E}_{\mathbf{x}}[(\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x}))^2]}_{\text{Viés}} \quad (7.1)$$

Posto de maneira simples, o erro (MSE) esperado pode ser decomposto em dois termos. O primeiro está relacionado com o quanto $h_{\mathcal{D}}$ pode variar caso re-amostremos o conjunto de treino $\mathcal{D}$. O segundo leva em consideração o quadrado do viés de $h_{\mathcal{D}}$ para cada $\mathbf{x} \sim \mathbb{P}_{\mathbf{x}}$. Vale adicionar que é possível derivar resultados similares para outras escolhas de ℓ . Para um tratamento teórico mais rigoroso, o leitor pode recorrer a livros como *Understanding machine learning: from theory to algorithms*, por Shalev-Shwartz e Ben-David (2014).

Como isso muda nossa vida? Na maioria das aplicações, temos pouca informação sobre f . Nesse caso, nosso primeiro instinto talvez fosse escolher um método capaz de gerar aproximações arbitrariamente intrincadas. No entanto, métodos mais flexíveis costumam estar associados à uma maior variância no processo de aprendizado, o que influencia negativamente a performance esperada para novos dados de entrada. Por outro lado, métodos simples como regressão linear possuem baixa variância, mas podem apresentar alto viés caso f não possa ser bem aproximada por um (hiper-)plano. Em suma, não existe uma bala de prata. Precisamos de protocolos empíricos bem definidos para escolher o método mais adequado para cada situação.

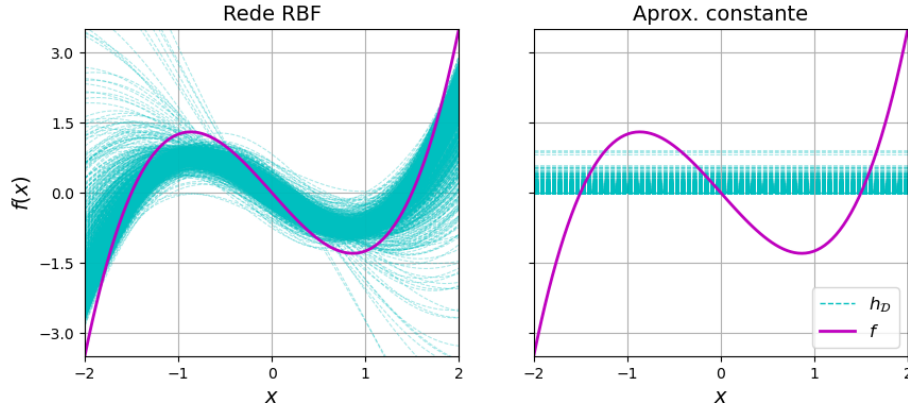


Figura 7.1: Ilustração do dilema viés-variância. A curva magenta representa $f(x) = (x+1)(x-1)x$ e cada curva em ciano representa a aproximação $h_{\mathcal{D}}$ aprendida para uma realização do conjunto de treino de tamanho 20, com vetores de entrada amostrados de uma uniforme com suporte $[-2, 2]$. Para este exemplo, comparamos redes RBF contra aproximações constantes, i.e., da forma $h(x) = c$ para algum $c \in \mathbb{R}$. Note que as curvas parametrizadas por redes RBF costumam capturar melhor o comportamento de f mais adequadamente que a aproximação constante. No entanto, também observamos uma variância maior também para a rede RBF.

Erros de generalização, treino e teste. Para computar o erro de generalização $E_{\text{gen}} = \mathbb{E}_{\mathbf{x}}[\ell(h_{\mathcal{D}}(\mathbf{x}), f(\mathbf{x}))]$ de um modelo treinado $h_{\mathcal{D}}$ nós precisaríamos ter acesso à f e $\mathbb{P}_{\mathbf{x}}$. Com isso, poderíamos obter uma estimativa de Monte-Carlo $1/M \sum_{i=1}^M \ell(h_{\mathcal{D}}(\mathbf{x}_i), f(\mathbf{x}_i))$, com $\mathbf{x}_1, \dots, \mathbf{x}_M \sim \mathbb{P}_{\mathbf{x}}$. No entanto, raramente temos acesso a $\mathbb{P}_{\mathbf{x}}$ e $f(\cdot)$.

Uma solução simples consiste em dividir aleatoriamente o conjunto de observações disponíveis em subconjuntos de treino e teste, i.e., $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$. O conjunto $\mathcal{D}_{\text{train}}$ é naturalmente o que usamos para construir nossa aproximação $h(\cdot)$. O conjunto de teste $\mathcal{D}_{\text{test}}$ é usado para estimar o erro de generalização de $h_{\mathcal{D}_{\text{train}}}$ como

$$E_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x}, y) \in \mathcal{D}_{\text{test}}} \ell(h_{\mathcal{D}_{\text{train}}}(\mathbf{x}), y).$$

Note que E_{test} é uma aproximação de Monte-Carlo para E_{gen} e, desde que $\mathcal{D}_{\text{train}}$ e $\mathcal{D}_{\text{test}}$ sejam amostrados independentemente, é um estimador não-viesado de erro de generalização para $h_{\mathcal{D}_{\text{train}}}$. Para fins de diagnóstico, também é comum medir o erro no conjunto de treinamento:

$$E_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}, y) \in \mathcal{D}_{\text{train}}} \ell(h_{\mathcal{D}_{\text{train}}}(\mathbf{x}), y).$$

7.2 Underfitting e Overfitting

Dois fenômenos indesejáveis são recorrentes em aplicações de AM: underfitting e overfitting. Para facilitar nossa discussão, deixe que $\mathcal{H}$ denote a classe de modelos definida por um método. Por exemplo, para regressão linear, $\mathcal{H}$ é o conjunto das funções da forma $h(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}$.

Informalmente, underfitting ocorre quando não tentamos aproximar f usando uma classe de modelos pouco flexível $\mathcal{H}$, que tipicamente não contém f . Por exemplo, se $\mathcal{H}$ é a família das funções constantes, qualquer $h \in \mathcal{H}$ está fadado a ser uma má aproximação para $f(\mathbf{x}) = e^{-\|\mathbf{x} - \mathbf{c}\|_2^2} + e^{-\|\mathbf{x} - \mathbf{c}'\|_2^2}$, independente da quantidade de dados — veja o painel à esquerda na Seção 7.2. Nos casos de underfitting, costumamos observar erros altos tanto no conjunto de treino quanto no de teste.

Por outro lado, overfitting é caracterizado quando o modelo se sai muito pior no conjunto de teste que no de treino. O caso mais estereotípico de overfitting ocorre quando aproximamos uma função f simples usando uma classe de modelos muito mais complexa — por exemplo, quando f é linear mas $\mathcal{H}$ é o conjunto de polinômios de grau alto. No entanto, overfitting pode ocorrer mesmo quando as funções $h \in \mathcal{H}$ são mais simples que f quando os dados são poucos ou muito ruidosos. Os painéis mais à direita na Seção 7.2 ilustram esses casos.

7.3 Regularização

Uma maneira empírica de balancear o compromisso entre viés e variância é adotar uma classe de modelos suficientemente flexível e controlar a magnitude dos parâmetros dos modelos. Com isso, podemos limitar a capacidade de aproximação da classe de modelos escolhida.

Quando o processo de treinamento se resume a reduzir uma função de perda $\mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta})$, a praxe é adicionar à perda uma penalização que aumente com a magnitude dos parâmetros. A mais popular dessas penalizações é a regularização L_2 , que consiste em adicionar $c\|\boldsymbol{\theta}\|_2^2$ ao objetivo de aprendizado, onde c é uma constante positiva e $\boldsymbol{\theta}$ é o vetor de parâmetros. Para perceber que, à medida que c aumenta, $\arg \min_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}) + c\|\boldsymbol{\theta}\|_2^2$ se aproxima de $\mathbf{0}$. A Seção 7.3 ilustra sucintamente esses fenômenos. No paradigma Bayesiano, a regularização L_2 é similar a uma priori Gaussiana $\mathcal{N}(\mathbf{0}, 1/2c^{-1})$.

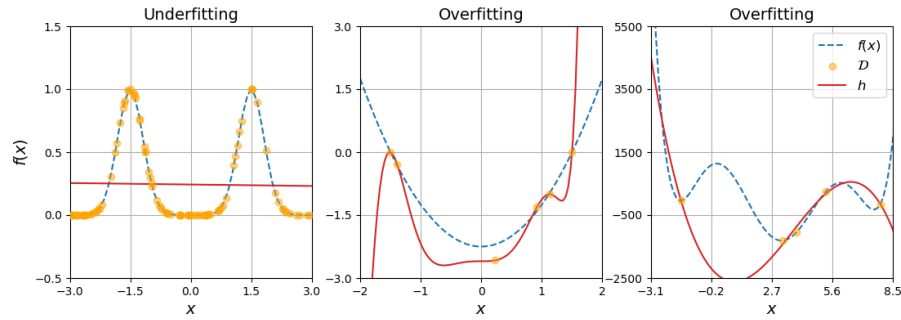


Figura 7.2: Overfitting vs. underfitting. (Esquerta) Quando $f(x) = e^{-\|x-1.5\|_2^2} + e^{-\|x+1.5\|_2^2}$, qualquer função linear h está fadada a ser uma má aproximação, independente da quantidade de dados que possuímos. (Centro) No caso em que $\mathcal{H}$ possui funções muito mais complexas que f , é possível que nos ajustemos excessivamente aos pontos vistos durante o treino, halucinando o que acontece em "regiões em branco". Nesse caso, estamos aproximando uma parábola com um polinômio de grau dez. (Esquerda) Mesmo que $\mathcal{H}$ contem apenas funções mais simples que f , ainda pode ocorrer overfitting. Nesse caso, f é um polinômio de grau seis, enquanto $\mathcal{H}$ contém apenas polinômios de grau três. Nesse caso, a falta de dados também pode nos levar à halucinar o formato de f em regiões não exploradas. Pela a escassez de dados, talvez beneficiássemos de um modelo com maior viés, como um polinômio de grau dois — que acabaria sendo ajustado com concavidade para cima.

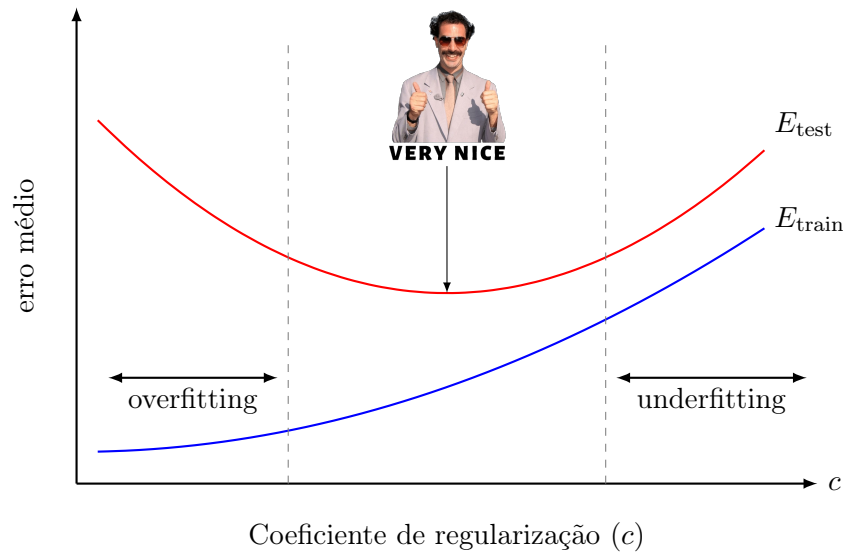


Figura 7.3: Ilustração do efeito da regularização L_2 sobre modelos altamente expressivos. Quando o coeficiente de regularização c é baixo, espera-se que h apresente baixíssimo E_{train} , mas alto E_{test} . Por outro lado, quando $c \rightarrow \infty$, tanto E_{train} quanto E_{test} se tornam demasiadamente altos. Em geral, estamos em busca de um valor intermediário para c que minimize E_{test} .

7.4 Seleção de hiperparâmetros

Até o momento, sabemos que podemos usar E_{test} para acessar a capacidade de generalização de dois algoritmos de aprendizado diferentes treinados em $\mathcal{D}_{\text{train}}$. No entanto, treinar um algoritmo envolve escolher hiperparâmetros. Resta-nos então uma pergunta: como escolher esses hiperparâmetros? Intuitivamente, gostaríamos de escolher hiperparâmetros que maximizem E_{test} , no entanto não podemos utilizar $\mathcal{D}_{\text{test}}$ durante o treino — caso contrário, comprometemos a noção de que E_{test} é uma estimativa do erro em novas amostras. No entanto, existe um problema fundamental nessa estratégia: podemos partir novamente o conjunto de treino, criando um novo conjunto $\mathcal{D}_{\text{val}}$ que chamamos conjunto de validação. Dessa maneira, podemos usar $\mathcal{D}_{\text{val}}$ para escolher hiperparâmetros, guardando $\mathcal{D}_{\text{test}}$ apenas para comparar a performance de diferentes classes de modelos. De maneira simples, podemos descrever esse processo através do algoritmo:

1. Particione $\mathcal{D}$ em $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{val}}$ e $\mathcal{D}_{\text{test}}$
2. Seja $\mathcal{M}$ uma lista com os métodos que desejamos considerar (e.g., k -NN, regressão linear)
3. Para cada método $M \in \mathcal{M}$:
 - (a) Seja $\mathcal{P}$ uma lista com os possíveis valores hiperparâmetros para M
 - (b) Para cada valoração de hiperparâmetro P :
 - i. Ajuste os parâmetros de M usando $\mathcal{D}_{\text{train}}$ e hiperparâmetros P
 - ii. Calcule o erro $E_{\text{val}}[P]$ do modelo aprendido no conjunto de validação
 - (c) Escolha $P^* = \arg \min_{P \in \mathcal{P}} E_{\text{val}}[P]$
 - (d) Treine M com hiperparâmetro P^* usando $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}}$
 - (e) Salve $P_{\text{opt}}[M] = P^*$
 - (f) Calcule o erro $E_{\text{test}}[M]$ em $\mathcal{D}_{\text{test}}$ do modelo treinado no passo anterior
4. Escolha o método ótimo $M^* = \arg \min_{M \in \mathcal{M}} E_{\text{test}}[M]$
5. Retreine M^* usando todos os dados disponíveis e $P_{\text{opt}}[M^*]$

Validação cruzada. Quando nosso conjunto de dados $\mathcal{D}$ é pequeno, o particionar em três pode fazer nossa estimativa do erro fora-de-amostra sofrer de alta variância, i.e., o modelo selecionado e sua performance podem variar drasticamente se usarmos uma partição ligeiramente diferente dos dados — mesmo que mantenhamos as proporções entre $|\mathcal{D}_{\text{train}}|$, $|\mathcal{D}_{\text{val}}|$ e $|\mathcal{D}_{\text{test}}|$. A ideia principal é adaptar o procedimento acima (da passo 1 ao 4) com diferentes partições de treino/validação/teste e escolher o modelo com menor erro de teste médio. Nesse caso, podemos definir o hiperparâmetro ótimo como aquele que gera menor erro de validação médio. Particularmente, partimos $\mathcal{D}$ em K conjuntos de tamanhos similares. Então, para cada repetição, escolhemos uma dessas K partições para ser $\mathcal{D}_{\text{test}}$ e as $K - 1$ restantes são usadas para definir $\mathcal{D}_{\text{train}}$ e $\mathcal{D}_{\text{val}}$. Chama-se esse processo de validação cruzada (*cross validation*). Note, no entanto, que o processo de escolha de hiperparâmetro dentro de cada repetição ainda pode sofrer de alta variância (já que $\mathcal{D}_{\text{val}}$ também é pequeno. Se quisermos reduzir ainda mais a variância do nosso procedimento, podemos usar um procedimento similar para definir os conjuntos de treino e validação K dado a partição de teste. Chama-se esse último procedimento de *nested cross validation*.

7.4.1 Parada precoce

Em algumas situações temos tantos dados que não precisamos nos preocupar em retreinar os modelos para incluir dados de validação ou teste após a seleção de hiper-parâmetros ótimos.

Quando esse é o caso e estamos treinando um modelo usando métodos de gradiente, é comum utilizar uma técnica chamada parada precoce (*early stopping*). Em suma, *early stopping* consiste em verificar se o erro de validação está diminuindo a cada $T > 1$ épocas e parar o procedimento caso a resposta seja negativa. Quando atingimos essa condição, retornamos os valores de parâmetros que exibiram o menor erro de validação até então. Para escolher hiperparâmetros, podemos repetir o procedimento para diferentes valores de hiper-parâmetros e retornar o modelo com menor erro de validação.